

Intent-Driven Spatial Search Engine

A Hybrid NL-to-SQL Architecture for Conversational Geographic Queries

BITS Pilani —CS F266 Study Project— Final Report

1. Introduction

1.1 Background

Digital mapping and navigational systems have become integral to everyday life, yet their underlying search architectures remain fundamentally constrained when processing human-centric, conversational queries. Currently, most mainstream mapping engines rely heavily on deterministic keyword filtering. While these systems excel at locating standardized addresses or exact proper nouns, they suffer from catastrophic spatial prioritization failures when users input descriptive, intent-driven queries.

A prime example of this architectural limitation is the *Local Intent Failure*. When a user searches for a location based on environmental characteristics—such as querying “*quiet places near me*” while located in Pilani—current systems fail to comprehend “*quiet*” as a desired spatial attribute (indicating low traffic, low noise, or a serene environment). Instead of parsing the semantic intent of the query, the search engine defaults to a rigid, exact-keyword match against its database of registered business names and landmarks.

Consequently, the system prioritizes exact word matches over geographic proximity. It surfaces highly irrelevant results, such as a business named “Quiet Place in HSR Layout” in Bengaluru or “Quiet Lands Gate” in Hyderabad. By returning locations over 1,000 kilometers away, the engine completely ignores the explicit spatial constraint established by the phrase “near me.” Such failures actively erode user trust. If a platform’s core “near me” functionality breaks down during natural language queries, users are highly likely to abandon the system for competitors. This disconnect demonstrates that the inability to process conversational spatial queries is not a superficial User Interface (UI) issue, but a deep, fundamental architectural problem.

In the Indian context, this architectural limitation is especially pronounced. Spatial references and local navigation are rarely confined to perfectly structured street addresses; they are highly descriptive, informal, and contextual. Users frequently search for places based on utility, vibe, or immediate physical needs—such as “late-night study cafes,” “safe parks for kids,” or “uncrowded restaurants.” When mapping engines treat these nuanced, multi-dimensional requests as flat text strings to be matched against formal business registrations, the resulting disconnect renders the digital map effectively useless for intuitive, local exploration.

Addressing this gap requires a paradigm shift in spatial search engines. There is a critical need to transition from rigid keyword-matching pipelines to advanced, intent-driven spatial modeling. The

system must learn to distinguish between a user looking for a place named “Quiet” and a user seeking the attribute of “quietness” within a tightly bounded local radius. This project focuses on that fundamental architectural transformation, aiming to map human-centric vocabulary to underlying geographic realities. By doing so, it will restore the integrity of core functionalities like spatial proximity and align digital map systems with how users actually perceive and navigate their physical surroundings.

1.2 Motivation

Despite the ubiquity of digital maps and navigational tools, their effectiveness remains deeply uneven when handling natural, human-centric spatial queries. There is a significant performance gap between structured, address-based searches and descriptive, intent-driven requests. This disparity is primarily driven by the fact that current mapping architectures are optimized for exact noun matching (e.g., specific business names or formal street addresses) rather than understanding the semantic characteristics or environmental attributes of a location.

Existing spatial search engines often struggle, particularly in regions like India, due to several unique geographic and cultural challenges. Spatial references in the Indian context are frequently unstructured, heavily reliant on informal landmarks, and expressed through colloquial descriptions rather than grid-based coordinates. Because current engines lack intent modeling, they exhibit extremely poor performance when users search for places based on utility, vibe, or immediate physical needs. As a result, highly relevant local spaces are ignored simply because their formal business registrations do not explicitly contain the user’s descriptive keywords.

Another critical motivation stems from the growing need for intuitive, localized navigational systems. Users rely on digital maps to access essential real-world services—such as finding a “late-night pharmacy,” a “quiet study space,” or a “safe park for kids.” When map platforms fail to understand the underlying semantic intent or ignore vital spatial constraints like “near me,” they create a tangible barrier to local discovery and everyday utility. Bridging this gap requires search architectures capable of mapping human vocabulary to physical realities, rather than forcing the user to guess the exact registered name of a place.

Furthermore, solving this problem does not simply require expanding existing keyword databases—a brute-force approach that is inefficient and fails to address the root semantic disconnect. This motivates the exploration of intelligent, intent-driven architectural alternatives. By creating a system that dynamically maps conversational queries to spatial attributes, we can achieve highly accurate local search results without relying on rigid, exact-match filtering.

Therefore, this project is motivated by the critical need to address the structural limitations of current spatial search engines. By transforming deterministic keyword filtering into robust spatial intent modeling, this work aims to develop a scalable, intuitive, and highly accurate navigational framework that truly reflects the dynamic, descriptive way people explore their physical surroundings.

1.3 Objective

The primary objective of this project is to architect an intelligent, intent-driven spatial search engine designed to bridge the structural gap between natural human language and formal geographic databases. By transitioning away from deterministic keyword filtering, the system aims to comprehensively capture the semantic intent underlying conversational queries. Rather than processing inputs as flat text strings, the architecture deconstructs multi-dimensional requests, mapping human-centric vocabulary to the underlying environmental attributes and implicit spatial constraints perceived by the user.

To realize this architectural shift, the project focuses on four core technical pillars:

- **Geospatial Database & Ontology Construction:** Designing a robust, denormalized relational framework using PostgreSQL and PostGIS to model physical locations, integrated with a Semantic Intent Ontology that maps colloquial expressions to structured geographic attributes.
- **Dynamic Intent Capturing & NL-to-SQL Translation:** Optimizing specialized Text-to-SQL models (such as Defog SQLCoder) to extract explicit and implicit intents from unstructured queries, ensuring subjective user needs are translated into executable, strictly bounded spatial commands.
- **Rigorous Hybrid Evaluation:** Developing a custom Hybrid Evaluation Framework that combines Abstract Syntax Tree (AST) structural parsing with live database execution to empirically validate model accuracy and semantic performance against real-world spatial entropy.
- **Conversational Interaction Layer:** Implementing an intuitive, multi-modal interface capable of managing spatial context and follow-up inputs to capture evolving user intents, synthesizing raw database outputs into localized navigational dialogue.

2. Related Work (with Comparisons)

2.1 Introduction

The development of intuitive spatial search engines has gained significant attention due to the growing gap between how humans colloquially describe locations and how geographic databases formally store them. While mainstream commercial map platforms demonstrate strong capabilities in standard address routing and exact-name lookups, they frequently suffer from catastrophic spatial prioritization failures when handling descriptive, intent-driven queries. This limitation primarily arises from an over-reliance on deterministic keyword matching against business titles, an inefficient mapping of conversational vocabulary to structured spatial categories, and a failure to mathematically enforce local bounding boxes during semantic searches. This section reviews existing spatial search architectures and highlights the critical need for intent-based spatial routing.

2.2 Traditional Geocoding and Commercial Mapping Engines

The most common approach to spatial search relies on hierarchical indexing and Information Retrieval (IR) techniques.

- **Traditional Geocoders (e.g., Nominatim):** Nominatim, the primary search engine for OpenStreetMap (OSM), utilizes a highly structured spatial index (country, state, city, street, house number). While it offers high precision for exact addresses and structured administrative boundaries, its architecture is strictly deterministic. It requires rigid input and is semantically blind, meaning it fails completely on descriptive queries such as “quiet places” or “late-night cafes.”
- **Commercial Mapping Engines (e.g., Google Maps, MapmyIndia/Mappls):** Industry-standard platforms utilize massive, proprietary Point of Interest (POI) databases combined with text-ranking algorithms (like TF-IDF or Elasticsearch). While they excel at locating registered businesses and incorporating user reviews, their fundamental architecture is flawed for natural language. They suffer from Exact-Match Bias, heavily weighting exact word matches in a business’s registered title over geographic proximity. Consequently, a descriptive search for “quiet places near me” in Pilani will often surface a registered business named “Quiet Place” in Bengaluru (1,000+ km away), breaking the spatial constraint of the user’s intent. They treat queries as flat text strings rather than extracting multi-dimensional intents (utility + environment + spatial bounds).

2.3 Semantic POI Recommendation Systems

To address the limitations of deterministic keyword matching, recent academic systems have attempted to improve spatial search using semantic embeddings and Graph Neural Networks (GNNs).

- **Graph-Based and Hybrid Recommenders:** These systems model relationships between user preferences, past behavior, and venue tags to recommend POIs. They excel at understanding category synonyms and capturing the “vibe” of a location that traditional geocoders miss. However, they possess a major structural limitation: Weak Spatial Enforcement. These models often rank results by semantic text similarity first, treating spatial proximity as a secondary ranking signal rather than a hard constraint. Consequently, they struggle to strictly enforce geographic boundaries (like a rigid 5-kilometer bounding box) and are notoriously difficult to integrate directly with raw relational spatial databases like PostGIS for real-time, low-latency querying.

2.4 Research Gaps & Proposed Advantages

A review of current spatial search architectures reveals three major research gaps:

- **Exact-Match Bias (Name vs. Attribute):** Systems prioritize exact text matches in formal business titles, leading to catastrophic routing failures when users search for environmental attributes.
- **Deterministic Filtering:** An inability to dynamically map colloquial, human-centric needs (e.g., “uncrowded,” “safe,” “study spot”) to formal geographic tags.
- **Decoupled Architecture:** A fundamental disconnect between natural language processing and the underlying relational spatial database, resulting in spatial constraints being abandoned in favor of semantic text matching.

What This Project Achieves Better: This project bridges these critical gaps by fundamentally shifting the search architecture from rigid keyword filtering to *intent-based spatial routing*:

- **Intent Translation Over Keyword Matching:** Instead of executing a flat text search for a business named “Quiet,” the system translates the conversational query into a structured, multi-dimensional filter. It deconstructs the user’s descriptive language and maps it against underlying environmental and utility attributes (e.g., translating “quiet” to specific venue tags like libraries, parks, or low-traffic areas).
- **Bridging Colloquial Vocabulary:** The architecture closes the gap between human speech and map data by utilizing a semantic ontology. The system understands informal, multi-dimensional requests (utility + vibe) and successfully maps them to formal geographic schemas.
- **End-to-End Database Routing:** By directly mapping natural language into structured, executable spatial queries (such as PostGIS SQL commands), this system ensures that semantic intent is natively understood by the database. This allows complex, descriptive constraints—alongside strict geographic boundaries—to be executed simultaneously at the data layer, ensuring high precision, low latency, and true local relevance.

3. Proposed Technique(s) and Algorithm(s)

This section outlines the proposed methodology for developing the intent-driven spatial search engine. The core of this architecture relies on converting natural language geographic queries into highly structured, executable spatial SQL (PostGIS) commands. To achieve this, the project required selecting an optimal translation model and, more importantly, developing a robust evaluation framework to guarantee the model’s accuracy on complex spatial databases.

3.1 Architectural Evolution and Model Selection

The initial conceptualization of the Natural Language to SQL (NL-to-SQL) pipeline utilized large, API-dependent general-purpose models, specifically starting with Llama-3-70B-Versatile. While this model demonstrated strong reasoning and could generate complex spatial queries, its implementation was fundamentally flawed for a scalable mapping engine. The reliance on external APIs introduced

prohibitive latency, high token costs, and a dependency on proprietary keys, rendering it unviable for local, real-time spatial routing.

To address these scalability and cost bottlenecks, the architecture was pivoted toward a specialized, open-source alternative: **Defog SQLCoder**. SQLCoder is a state-of-the-art model explicitly fine-tuned for Text-to-SQL tasks. It offers exceptional performance in database schema understanding and query generation while being lightweight enough to deploy locally. Local deployment is critical for spatial architectures, as it ensures data privacy, eliminates network-induced latency during query generation, and allows for deep integration with local geographic databases. However, before fully integrating SQLCoder into the spatial routing pipeline, it was imperative to rigorously benchmark its capability to handle highly denormalized, spatially-rich databases (such as OpenStreetMap-style tables utilizing PostGIS).

3.2 Evaluation Strategy Formulation

Evaluating NL-to-SQL systems is notoriously difficult, particularly in the geographic domain. A query can be syntactically correct but logically flawed, or syntactically entirely different from a “gold standard” but logically perfect. In highly denormalized spatial databases—where a single location might have dozens of overlapping tags (e.g., a building acting as a “cafe,” a “library,” and a “university”)—traditional benchmarking falls short. To evaluate the Defog SQLCoder model, we analyzed two leading academic frameworks for Text-to-SQL evaluation and identified their respective limitations.

3.2.1 Component-Based Structural Matching (The “Spider” Approach)

The first baseline evaluation metric considered was based on the methodology outlined in Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing (Yu et al., 2018).

- **Methodology:** The Spider framework avoids direct string matching, which is highly fragile, and instead uses “Exact Set Match.” It parses both the predicted query and the gold query into their underlying structural components (e.g., SELECT, WHERE, GROUP BY, ORDER BY) and checks if the sets of columns and operators match.
- **Limitation (High False Negatives):** While highly structured, this approach penalizes models for writing queries that achieve the correct result using a different, but equally valid, structural logic. In spatial databases, there are often multiple ways to filter for a location attribute. Punishing the model for structural deviation leads to a high rate of False Negatives—rejecting queries that would have successfully returned the correct spatial locations.

3.2.2 Execution-Based Semantic Matching (The “Test Suite” Approach)

To counter structural rigidity, the second framework analyzed was from Test Suite Accuracy for Text-to-SQL (Zhong et al., 2020).

- **Methodology:** This approach relies entirely on database execution. It runs the predicted query against multiple distilled databases or database states. If the output rows of the predicted query perfectly match the output rows of the gold query, it is marked as correct.
- **Limitation (High False Positives):** In highly denormalized databases, a poorly constructed query might accidentally return the correct result simply due to data distribution. For example, if a localized database only has one hospital, a vaguely constructed query searching for “all large buildings” might accidentally return the exact same single row as the gold query searching specifically for “hospitals.” This leads to a high rate of False Positives—accepting logically flawed queries based on lucky execution results.

3.3 The Proposed Hybrid Evaluation Framework

To eliminate the respective False Positives and False Negatives of the academic baselines, this project proposes and implements a **Hybrid Evaluation Framework**. This framework merges AST-based (Abstract Syntax Tree) structural testing with execution-based semantic testing to rigorously validate the SQLCoder model on spatial intents.

3.3.1 Golden Dataset Creation

A custom benchmark suite of 50 spatial queries was manually authored to reflect the nuances of the database. These queries represent real-world user intents in the Indian context (e.g., “Find late-night cafes near me,” “Quiet study spots,” or “Places that accept UPI”). Each entry contains the natural language question, a difficulty classification (Easy, Medium, Hard, Extra), the target intent cluster, and the manually verified “Gold SQL” query written to PostGIS standards.

3.3.2 AST-Based Structural Testing (using sqlglot)

To ensure the model fundamentally understands the database schema, the framework uses the sqlglot library to parse both the generated query and the Gold SQL into an Abstract Syntax Tree.

- The system mathematically normalizes table aliases and extracts specific operational components: SELECT columns, FROM tables, WHERE columns, WHERE operators (e.g., ILIKE, =, IS NULL), ORDER BY clauses, and LIMIT constraints.
- A structural score is calculated based on the intersection of these sets. This structural verification ensures the model is logically querying the correct spatial tags rather than hallucinating table columns.

3.3.3 Execution-Based Semantic Testing

Simultaneously, the framework connects directly to the local PostgreSQL/PostGIS database via SQLAlchemy to test empirical accuracy.

- The framework dynamically strips arbitrary constraints (like LIMIT 5) from both the Gold and Predicted queries. This is a crucial step to prevent alphabetical truncation from causing false mismatches in large geographic result sets.
- Both queries are executed, and the resulting geographical entities (target locations) are extracted into normalized, mathematical sets.
- An F1-Score is calculated based on the precision and recall overlap between the Gold result set and the Predicted result set. Given the immense entropy and overlapping tagging standards in real-world spatial data, an F1-Score of 0.50 or higher is utilized as the threshold for a successful semantic match. A 50% overlap strongly indicates that the core spatial intent was captured, forgiving minor data variations or fringe point-of-interest inclusions.

3.3.4 Final Hybrid Verification

The final algorithm merges the structural and semantic scores to output a definitive classification for every query. This multidimensional scoring provides deep insights into how the model operates:

- **PASS:** The query achieves a structural score of 0.50 or higher AND a semantic F1-score of 0.50 or higher. This indicates the query is both logically sound and empirically accurate, rendering it safe for production routing.
- **STRUCT_ONLY:** The query uses the correct columns, tables, and operators, but fails to execute properly or returns incorrect geographic locations. This categorization successfully catches False Positives, revealing when a model understands the schema but fails to filter the data correctly (e.g., using the wrong string value inside an ILIKE clause).
- **SEM_ONLY:** The query returns the correct geographic locations but utilizes unconventional or partially incorrect SQL syntax. This catches False Negatives from the Spider methodology, highlighting instances where the model found a functional “hack” or where the human-authored Gold SQL was overly complex.
- **FAIL:** The query fails both structural parsing and semantic execution. This indicates a complete breakdown in translation, requiring further prompt refinement.

3.4 The Conversational Interaction Layer

While the core backend relies on the rigorous translation of natural language to PostGIS SQL, exposing raw database execution to the end-user is impractical. To bridge the gap between complex spatial routing and user experience, the architecture incorporates a **Conversational Interaction Layer**. This serves as the dynamic, user-facing frontend of the spatial search engine, allowing users to interact with the map intuitively without needing to understand the underlying geographic ontology.

This interaction layer is designed around three primary architectural functions:

3.4.1 Dynamic Input Parsing and State Management

Unlike a traditional search bar that accepts static keywords, the interaction layer is designed to accept unstructured, conversational inputs (e.g., “I need a quiet place to study right now”). The interface captures this input and maintains a “spatial state” or context window. If the user follows up with a refinement—such as “What about ones that are open late?”—the interaction layer retains the original spatial constraints (quiet, study) and dynamically appends the new utility constraint (late-night) before passing the revised prompt to the SQLCoder model for re-routing.

3.4.2 Data-to-Dialogue Translation

When the PostGIS database successfully executes the generated spatial query, it returns raw relational data (e.g., geographic coordinates, venue names, distance metrics, and utility tags). The interaction layer is responsible for translating this raw data array back into a natural, conversational response. Instead of simply dumping a list of names, the interface synthesizes the results contextually. For example, it translates the raw database output into a fluid response: “I found 3 quiet study spaces within a 2-kilometer radius. The closest is the Central Library, but The Reading Room is open until midnight.”

3.4.3 Multi-Modal Spatial Synchronization

A critical feature of this interface is the synchronization between the conversational dialogue and the visual map UI. The interaction layer does not operate as a standalone text box; it is deeply coupled with the map rendering engine. When the system converses with the user about specific locations, it simultaneously plots the corresponding geographic coordinates, adjusts the map zoom to frame the exact spatial bounding box of the query, and highlights the recommended routes. This multi-modal feedback loop ensures that the user’s conversational intent is immediately reflected in their visual geographic reality, creating a seamless, trust-building navigational experience.

3.5 Offline Deployment via llama.cpp and 4-Bit Quantization

To eliminate the remaining cloud dependency and achieve full offline operation on a campus laptop, the SQLCoder weights were re-deployed locally using **llama.cpp** with **4-bit GGUF quantization**, and a 2B-parameter **Kimi** grammar parser was placed in front of the Text-to-SQL stage as the conversational front end. The motivation was threefold: (i) removing API latency during query generation; (ii) eliminating per-token cost and proprietary key dependencies; and (iii) ensuring data privacy in environments where the campus Wi-Fi is unreliable. With INT4 weight quantization the SQLCoder model fits into a fraction of its original VRAM footprint, allowing it to run end-to-end on commodity local hardware. The resulting pipeline—Kimi 2B parser → quantized SQLCoder → PostGIS—operates entirely without any network call.

Two trade-offs emerged from this deployment. First, INT4 quantization is known to compress the precision of weights to such a degree that schema-sensitive layers (those responsible for column-name selection and JOIN-clause construction) occasionally degrade. Second, the 2B Kimi parser offers a much

smaller grammatical decomposition capacity than the original conversational front end, producing under-specified prompts on the more nuanced spatial intents. Both effects are revisited empirically in Section 5.

4. Data Set(s) Used with Description

The efficacy of an intent-driven spatial search engine relies on two foundational data components: a robust, highly denormalized geographic database representing the physical world, and a rigorously curated benchmark dataset that maps natural language conversational queries to executable spatial SQL. Because standard public Text-to-SQL datasets lack domain-specific geographic nuances, spatial operators, and localized Indian context, a custom dual-layered data architecture was developed specifically for this project.

4.1 The Spatial Database (Geographic Ground Truth)

To simulate a real-world, highly complex mapping environment, the underlying data was sourced from OpenStreetMap (OSM) and ingested into a locally hosted PostgreSQL database equipped with the PostGIS extension. The data focuses on the Pilani region, serving as a bounded but feature-rich geographic testing ground.

Rather than using clean, highly normalized relational tables, the database is intentionally structured as a highly denormalized flat table (`pilani_places`) to accurately reflect the chaotic, tag-heavy nature of real-world GIS systems. Real-world spatial data is notoriously asymmetrical, frequently requiring the engine to infer constraints based on inconsistent or one-sided data shapes rather than perfectly balanced columns.

The schema categorizes data into distinct feature sets:

- **Core Identifiers:** OSM IDs, official names, alternate names, and multilingual tags.
- **Geospatial Data:** PostGIS geometry columns for spatial bounding and distance calculations.
- **Primary Place Categories:** High-level geographic entities such as amenity (hospital, library, cafe), building (dormitory, residential), shop, leisure, and sport.
- **Utility & Attribute Tags:** Granular metadata columns crucial for intent matching, including `opening_hours`, `indoor`, `lit`, `surface`, and payment infrastructures like `payment:upi`.

4.2 Semantic Intent Ontology (Vocabulary Mapping)

Formal geographic databases do not store human subjective experiences or utility-based colloquialisms. Therefore, a structured mapping dictionary was authored to serve as the connective tissue between user vocabulary and the database schema.

This ontology translates abstract human intents into concrete SQL filtering rules:

- **“Late Night” Intent:** Maps to opening_hours ILIKE ‘%24/7%’ OR ‘%22:%’ OR ‘%23:%’ OR ‘%00:%’.
- **“Study” Intent:** Maps to amenity ILIKE ‘%library%’ OR building ILIKE ‘%university%’.
- **“Photogenic / Vibe” Intent:** Maps to a combination of tourism, historic, artwork_type, or man_made columns.
- **“Digital Payment” Intent:** Maps to exact boolean flags like “payment:upi” = ‘yes’.

This ontology was injected directly into the system’s architecture, ensuring the translation model possessed the domain knowledge to route subjective queries to objective data points.

4.3 The Golden Benchmark Suite (Evaluation Dataset)

To rigorously test the architecture’s ability to execute intent-based spatial routing, a custom “Golden Benchmark Suite” was authored. This dataset consists of 50 hand-crafted, contextually relevant spatial queries explicitly designed to test the boundaries of semantic translation in an Indian context.

To gauge the model’s analytical depth, the benchmark dataset stratifies queries into four levels of increasing complexity: Easy, Medium, Hard, and Extra Hard. Each entry in the dataset pairs the natural language question with a manually verified, structurally perfect Gold SQL query. This custom dataset serves as the empirical ground truth for the Hybrid Evaluation Framework, allowing for the precise measurement of the system’s ability to translate messy, colloquial human requests into strict, executable geographic rules.

The custom Golden Benchmark Dataset created for this evaluation can be accessed here: [Golden Benchmark Suite](#)

5. Experiment(s) and Result(s)

To validate the efficacy of the intent-based spatial search architecture, an extensive empirical evaluation was conducted across two deployment configurations. Configuration A is the cloud-served reference: Defog SQLCoder-8B accessed through hosted inference, processing the custom 50-query Golden Benchmark Suite. Configuration B is the fully offline deployment: the same SQLCoder weights re-deployed locally via llama.cpp with 4-bit GGUF quantization, paired with a 2B-parameter Kimi grammar parser as the conversational front end.

In both configurations, the generated SQL outputs were subjected to the Hybrid Evaluation Framework (v4.1), which independently calculated an AST-based Structural Score (via sqlglot) and an execution-based Semantic F1-Score directly against the local PostgreSQL/PostGIS spatial database. A query was deemed a successful “Pass” only if it achieved both a structural score of ≥ 0.50 (indicating logical soundness) and a semantic F1-Score of ≥ 0.50 (indicating empirical accuracy against real-world data entropy). To ensure rigorous semantic validation, the framework dynamically stripped arbitrary

constraints (such as LIMIT 5) before execution, comparing the entirety of the generated and gold result sets rather than truncated samples.

5.1 Evaluation and Result(s) (Comparisons)

The proposed architecture demonstrated robust translation capabilities under Configuration A, successfully mapping conversational vocabulary to structured geographic constraints in the vast majority of test cases.

Summary of Global Metrics (Configuration A):

- **Total Queries Executed:** 50
- **PASS:** 42 (84.0%)
- **FAIL:** 1 (2.0%)
- **STRUCT_ONLY:** 7 (14.0%)
- **SEM_ONLY:** 0 (0.0%)
- **NO_PRED:** 0 (0.0%)

Average Scoring (Configuration A):

- **Average Structural Score:** 0.827
- **Average F1 Semantic Score:** 0.784

The system achieved a highly commendable 84.0% overall pass rate. The high Average Structural Score (0.827) indicates that the translation model excels at schema linking—it consistently identifies the correct tables, columns, and logical operators required to satisfy the user’s intent without hallucinating invalid database schema.

Crucially, the evaluation framework flagged 7 STRUCT_ONLY edge cases. In these instances, the model wrote syntactically valid SQL that perfectly matched the database structure but failed the semantic execution. It is important to note that these were not complete failures; rather, they resulted in partial semantic matches. Because our evaluation framework compares the entirety of the generated result set against the entirety of the gold result set, these queries failed because their partial overlap was insufficient to cross the 0.50 F1-Score threshold. This validates the necessity of the hybrid framework: had the system relied solely on static structural analysis, or evaluated only a limited top-5 sample, these queries might have been recorded as false positives.

5.1.1 Performance by Query Complexity (Spider-Style Analysis)

To understand the limitations of the translation layer, results were analyzed across the four stratified difficulty levels. The degradation of the F1-Score across complexity tiers highlights the inherent challenges of highly denormalized geographic data.

Difficulty	Total Queries	Passed	Pass Rate (%)	Avg Structural Score	Avg Semantic F1-Score
Easy	19	17	89.5%	0.851	0.842
Medium	16	14	87.5%	0.844	0.825
Hard	12	9	75.0%	0.750	0.708
Extra	3	2	66.7%	0.889	0.500

Observations by Complexity:

- **Easy & Medium (Direct & Multi-Column Intents):** The system excels at direct attribute lookups (e.g., mapping “parks” to leisure ILIKE ‘%park%’), maintaining pass rates near 90% and F1-Scores well above 0.80. The semantic ontology mapping successfully bridges straightforward colloquial vocabulary with backend database tags.
- **Hard (Asymmetrical Tagging & Negation):** Performance begins to drop (75% pass rate) when the model encounters asymmetrical data schemas, such as filtering for boolean flags that lack direct column equivalents, or executing strict negations (e.g., finding buildings that are not places of worship). The structural score drops to 0.750, reflecting the model struggling to construct complex IS NULL OR ... NOT ILIKE logic branches.
- **Extra Hard (Colloquial Location Splitting):** At the highest complexity level, the model’s structural score rebounded to 0.889, but the F1-Score plummeted exactly to the passing threshold of 0.500. This indicates that while the model wrote perfectly structured SQL, it struggled to capture the entirety of the required semantic subset when parsing highly informal, multi-part keyword strings.

5.1.2 Configuration B: Offline Deployment (llama.cpp + 4-Bit Quantized SQLCoder + Kimi 2B)

Configuration B re-deploys the same SQLCoder weights on commodity hardware via llama.cpp using 4-bit GGUF quantization, with the 2B-parameter Kimi parser handling conversational front-end grammar. This configuration achieves the project’s offline-first goal: zero API latency during query generation, no per-token cost, no proprietary key dependency, and full data privacy even when campus Wi-Fi is unavailable.

However, two regressions were observed against the Configuration A baseline. First, **INT4 quantization** introduced precision loss in the model’s schema-linking layer; the quantized network occasionally produced near-correct but subtly wrong column names or malformed JOIN clauses on Hard and Extra Hard queries—classes already sensitive to logical-grouping errors. Second, the **2B-parameter Kimi parser** exhibited weaker grammatical decomposition than the original conversational front end, producing

under-specified prompts that propagated downstream into the Text-to-SQL stage and amplified the partial-match problem already documented in Configuration A.

The net effect was an aggregate accuracy reduction concentrated in the Hard and Extra tiers, while the Easy and Medium tiers remained largely robust because their structural simplicity tolerates moderate precision loss. These observations are consistent with the general literature on aggressive quantization of code- and SQL-specialized LLMs, and they motivate the future-work agenda described in Section 6. Importantly, the offline configuration remains fully functional end-to-end: the regression is in accuracy, not in operability, and the trade-off (full offline operation in exchange for a measured accuracy delta on the hardest queries) is acceptable for a campus-deployable prototype.

5.2 Error Analysis

An analysis of the 1 total failure and 7 STRUCT_ONLY warnings reveals two primary architectural limitations that warrant future optimization. Both limitations are amplified under Configuration B because the smaller front-end parser produces more under-specified prompts, and INT4 weight precision degrades the schema-linking layer responsible for catching them.

- **Over-Aggressive Logic Grouping (The Partial Match Problem):** The majority of missed semantic targets occurred because the model applied AND logic where implicit OR logic was contextually required by human speech. When a user asks for “volleyball or table tennis courts,” the model occasionally attempts to find physical locations that possess both tags simultaneously. While this returns a few valid partial matches, comparing the whole result sets exposes that it artificially narrowed the geographic bounding box, dragging the F1-Score below 0.50.
- **String Matching Fragility:** In cases involving highly specific local landmarks (the “Extra” difficulty tier), the model occasionally relies on overly rigid string matching. If a user asks for “Meera Bhawan girls hostel,” the spatial database might only list it as “Meera Bhawan.” While the semantic ontology helps with generic categories (like “quiet” or “study”), the system still struggles with entity-resolution for unregistered colloquial nicknames of specific buildings, resulting in incomplete semantic recall.

Despite these edge cases, the experiment proves that replacing flat text-indexing with an intent-based NL-to-SQL architecture yields highly accurate, strictly bounded, and semantically aware geographic search results 84% of the time under Configuration A, with a measured and predictable degradation under the offline Configuration B.

6. Conclusion and Future Work

This project demonstrates that the catastrophic spatial prioritization failures of mainstream mapping engines on conversational queries are not a UI defect but a deep architectural one, and that an intent-driven NL-to-SQL routing pipeline coupled with a Semantic Intent Ontology and a Hybrid

AST-plus-execution evaluation framework can replace flat keyword indexing for localized search. The cloud-served reference configuration (Defog SQLCoder-8B) achieves an 84.0% pass rate on a stratified 50-query benchmark, with an average structural score of 0.827 and an average semantic F1 of 0.784. The fully offline configuration based on llama.cpp 4-bit quantization paired with a 2B Kimi parser is functional end-to-end but regresses in accuracy due to quantization-induced precision loss in the schema-linking layer and weaker grammatical front-end parsing.

Future work focuses on closing this regression gap and hardening the system for real-world campus deployment:

- **Targeted-Precision Quantization:** Selectively de-quantize the schema-linking layers of the offline SQLCoder while keeping general-purpose layers in INT4 to recover Hard-tier accuracy without sacrificing the VRAM benefits of quantization.
- **Larger On-Device Conversational Front End:** Replace the 2B Kimi parser with a 7B-class on-device front end (or a fine-tuned grammar-specialized variant) to reduce upstream prompt under-specification.
- **Prompt Refinement for Disjunctive Logic:** Refine the system prompt to enforce OR-grouping in cases of disjunctive utility intents (“volleyball or table tennis courts”), eliminating the Over-Aggressive Logic Grouping pattern.
- **Ontology Expansion for Entity Resolution:** Expand the Semantic Intent Ontology with localized building aliases and informal hostel nicknames to improve entity resolution on the Extra-difficulty tier.
- **End-to-End Latency Optimization:** Tune end-to-end latency between user input and PostGIS response so the offline system is competitive with the cloud reference for real-time campus use.
- **Custom Conversational Interaction Layer:** Replace the current generic LLaMA-based chat front end with a spatial-search-specific Conversational Interaction Layer that maintains spatial state across follow-up turns and synthesizes raw PostGIS rows into helpful natural-language responses.

7. Work Update

7.1 Work Completed

To date, the foundational architecture of the intent-driven spatial search engine has been successfully designed, implemented, and rigorously evaluated. The project has moved beyond the theoretical phase into a functional, end-to-end routing prototype operating in two configurations (cloud reference + fully offline).

The specific milestones achieved include:

- **Database & Ontology Construction:** A robust geographic ground-truth database for the Pilani region was successfully deployed using PostgreSQL and PostGIS. This included the manual ingestion of OpenStreetMap (OSM) data and the creation of a Semantic Intent Ontology to map colloquial human descriptors (e.g., “quiet,” “vibe,” “late-night”) to formal database tags.
- **Benchmark Dataset Creation:** A highly stratified, 50-query Golden Benchmark Suite was curated. This dataset establishes an empirical baseline for spatial intent testing, covering various geographic clusters including Food, Academics, Residential, and Logistics.
- **Core Model Identification & Integration:** After evaluating several large-scale, API-dependent models, Defog SQLCoder-8B was identified as the optimal engine. It was integrated into a local environment to ensure low-latency query generation and total data privacy, bypassing the need for expensive external API keys.
- **Rigorous Architectural Testing:** The custom Hybrid Evaluation Framework was developed and executed. This framework utilized Abstract Syntax Tree (AST) parsing for structural logic and live database execution for semantic accuracy. Using a 0.50 F1-Score threshold to account for real-world data entropy, the system achieved a highly successful 84.0% overall pass rate (Configuration A).
- **Offline Deployment via llama.cpp + 4-bit Quantization + Kimi 2B:** The SQLCoder weights were re-deployed locally with 4-bit GGUF quantization through llama.cpp, paired with a 2B Kimi grammar parser. This delivered a fully offline, end-to-end pipeline (Configuration B) operable on commodity hardware without any network call. The accuracy regression observed in this configuration has been characterized and traced to two specific causes (quantization-induced precision loss in schema linking, and limited grammatical capacity in the smaller parser), as documented in Section 5.1.2.
- **Prototype Frontend Integration:** A functional prototype of the search engine has been deployed. Currently, this prototype uses a generic LLaMA model to act as the conversational frontend. This initial integration allows for basic chatbot interaction, proving that natural language can indeed be routed through a specialized SQL engine to return localized geographic results.

7.2 Work Remaining

While the backend routing, semantic translation pipelines, and offline deployment are functional, the transition from a generic prototype to a specialized production system requires several key developments. The remaining work is focused on the following areas:

- **Targeted-Precision De-Quantization of Configuration B:** To recover the Hard- and Extra-tier accuracy lost to aggressive INT4 quantization, the schema-linking layers of the offline SQLCoder will be selectively held at higher precision (e.g., INT8 or FP16) while keeping general layers in

INT4. This is expected to close most of the gap to the cloud reference without losing the VRAM-footprint benefit.

- **Upgrading the Offline Conversational Front End:** The 2B Kimi parser will be replaced with a 7B-class on-device front end (or a fine-tuned grammar-specialized variant) to reduce upstream prompt under-specification that currently amplifies the partial-match problem.
- **Development of a Specialized Custom Chatbot:** The current reliance on a general LLaMA model for user interaction is a temporary measure. The primary remaining task is to develop a custom Conversational Interaction Layer tailored specifically for spatial search. This involves creating a system that can manage “spatial state”—remembering that a user is looking for a “quiet” place even when they ask a follow-up question like “are there any open right now?”
- **Advanced Data-to-Dialogue Synthesis:** Beyond simply finding the data, the custom chatbot must be trained to explain why certain results were chosen. Work remains to build a synthesis engine that takes raw PostGIS rows and turns them into helpful, natural responses (e.g., “I found two libraries that are quiet, but one has better seating for long study sessions”).
- **Prompt Refinement and Edge-Case Resolution:** Analysis of the current 16% failure/partial-match rate shows that the NL-to-SQL prompt requires further tuning. Remaining work includes optimizing the model’s logic to prevent “Over-Aggressive Logic Grouping”—ensuring the model doesn’t use AND logic when a user clearly intends an OR relationship between attributes.
- **Ontology Expansion for Entity Resolution:** To improve the “Extra Hard” query pass rate, the semantic ontology must be expanded to include more informal nicknames and localized building aliases. This will reduce string-matching fragility and ensure the system can resolve nicknames of hostels and departments that may not be formally registered in the OSM database.
- **System Latency Optimization:** Final performance tuning will be conducted to minimize the time between a user submitting a conversational query and the PostGIS engine returning the final result, ensuring the system feels snappy and responsive for real-world use.

8. References

8.1 Project Development Artifacts

- **Weekly Progress Documentation:**
<https://docs.google.com/spreadsheets/d/1Nmng1ckKtnqWPiZjMQ5DJSJZmonJbVocLoX2OAqwtLw/edit?usp=sharing>
- **Initial System Prototype (LLaMA-Frontend):**
<https://colab.research.google.com/drive/1IMSdt-P4E1Cdwa9sjY9Mg8MiHmgrl9wc?usp=sharing>

- **Hybrid Evaluation Framework (The Tester):**
<https://colab.research.google.com/drive/1y3DBQz7fLRT1n-yGT4tOOLZ3eXeKyJUr?usp=sharing>
- **Current System Prototype:**
https://colab.research.google.com/drive/1DuhPBOBep57lI2cMS8_uDc3mEvALnDT6?usp=sharing
- **Golden Benchmark Suite (Evaluation Dataset):**
https://docs.google.com/document/d/1RUERl1EAD5a114_caHKXYTgNX1oANYGxytmjRxPf94/e/dit?usp=sharing

8.2 Research Foundations and Technical Platforms

- Spider Benchmark (Structural Matching): Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. <https://aclanthology.org/D18-1425.pdf>
- Test Suite Accuracy (Semantic Matching): Zhong, R., Yu, T., and Klein, D. (2020). Semantic Evaluation for Text-to-SQL with Distilled Test Suites. <https://aclanthology.org/2020.emnlp-main.29.pdf>
- Defog SQLCoder: Defog AI. SQLCoder: State-of-the-art LLM for Text-to-SQL Generation. <https://defog.ai/sqlcoder/>
- PostGIS Documentation: The PostGIS Development Group. PostGIS Spatial Database Management for PostgreSQL. <https://postgis.net/documentation/>
- sqlglot Parser: sqlglot: An Omni-dialect SQL Parser, Transpiler, Optimizer, and Engine. <https://github.com/tobymao/sqlglot>
- llama.cpp: Gerganov, G. llama.cpp: Inference of LLaMA model in pure C/C++. <https://github.com/ggerganov/llama.cpp>
- OpenStreetMap: OpenStreetMap contributors. OpenStreetMap. <https://www.openstreetmap.org/>
- Nominatim: Nominatim Development Team. Nominatim: Open-source geocoder for OpenStreetMap data. <https://nominatim.org/>